📚 جزوه سیستم عامل | فصل ۸: بن‌بست (Deadlock)

📝 بخش ۵: پیشگیری از بن‌بست (Deadlock Prevention)

🎯 هدف کلی پیشگیری

در روش "پیشگیری"، ما می‌خواهیم کاری کنیم که سیستم **هرگز** نتواند وارد بن‌بست شود. چطور؟ یادتان هست که گفتیم برای بن‌بست، ۴ شرط (انحصار متقابل، نگهداری و انتظار، عدم پس‌گیری، انتظار چرخشی) باید **همزمان** برقرار باشند؟

استراتژی پیشگیری: اگر کاری کنیم که **حتی یکی** از این ۴ شرط نتواند برقرار شود، بن‌بست غیرممکن می‌شود.

بیایید تک‌تک شرط‌ها را بررسی کنیم و ببینیم چطور می‌توانیم آن‌ها را از بین ببریم (نقض کنیم).

۱. حمله به شرط اول: انحصار متقابل (Mutual Exclusion) ⊘

- **ایده:** کاری کنیم که منابع قابل اشتراک باشند تا همه بتوانند همزمان استفاده کنند.
- **تحلیل:**
  - برای منابعی مثل "فایل‌های فقط خواندنی" (Read-only files) عالی است. (چون همه می‌توانند همزمان بخوانند و دعوا نمی‌شود).
  - **مشکل اصلی:** برخی منابع ذاتاً غیرقابل اشتراک هستند. مثلاً نمی‌شود یک "قفل" (Mutex) را همزمان به دو نفر داد. یا نمی‌شود همزمان دو چیز مختلف روی پرینتر چاپ کرد.
- **نتیجه:** این شرط را معمولاً **نمی‌توان نقض کرد**. ما مجبوریم انحصار متقابل را برای منابع غیر-اشتراکی حفظ کنیم.

۲. حمله به شرط دوم: نگهداری و انتظار (Hold and Wait) ⏳

- **ایده:** کاری کنیم که هیچ نخی همزمان که چیزی در دست دارد (Hold)، منتظر چیز دیگری نماند (Wait).

- **راه‌حل ۱ (همه یا هیچ):** نخ باید **قبل از شروع اجرا**، تمام منابعی که نیاز دارد را یکجا درخواست کند و بگیرد.
- **راه‌حل ۲ (دستان خالی):** نخ فقط زمانی اجازه دارد منبع جدید بخواهد که **هیچ منبعی در دستش نباشد.** (اگر چیزی دارد، باید اول آن را پس بدهد، بعد مجموعه جدید را بخواهد).
- **معایب بزرگ (نکته امتحانی):**
  1. **کاهش بهره‌وری (Low Utilization):** منابع زیادی بیکار می‌مانند. (مثلاً نخ، پرینتر را اول کار می‌گیرد ولی ۱۰ ساعت بعد لازمش دارد؛ در این مدت پرینتر بیهوده اشغال است).
  2. **قحطی‌زدگی (Starvation):** نخی که به منابع زیادی نیاز دارد، ممکن است هیچ‌وقت نتواند همه آن‌ها را همزمان خالی گیر بیاورد و تا ابد منتظر بماند.

۳. حمله به شرط سوم: عدم پس‌گیری (No Preemption) 🔄

- **ایده:** اجازه دهیم سیستم منابع را به زور از نخ‌ها بگیرد (Preempt کند).
- **راه‌حل:**
  - اگر نخی منبعی را بخواهد و مجبور شود صبر کند -> **باید تمام منابعی که الان دارد را آزاد کند (پس بدهد).**
  - لیست منابع مورد نیازش آپدیت می‌شود (منابع قبلی + منبع جدید). فقط وقتی دوباره اجرا می‌شود که **همه** را با هم بگیرد.
- **کاربرد:**
  - برای منابعی مثل **ثبات‌های CPU (Registers)** یا **تراکنش‌های دیتابیس** خوب است (چون می‌توان وضعیتشان را ذخیره و بازیابی کرد).
  - برای منابعی مثل **پرینتر** یا **قفل‌های Mutex** وحشتناک است! (نمی‌شود وسط پرینت کاغذ را کشید بیرون و بعداً ادامه داد).

۴. حمله به شرط چهارم: انتظار چرخشی (Circular Wait) 🔄🔥

((این عملی‌ترین و مهم‌ترین راهکار است))

- **ایده:** جلوی تشکیل "دایره انتظار" را بگیریم.
- **راه‌حل (ترتیب‌دهی منابع):**
  1. به هر نوع منبع یک **شماره منحصربه‌فرد** می‌دهیم. (تابع F).
     - مثلاً: نوارگردان = ۱، دیسک = ۵، پرینتر = ۱۲.

1. **قانون:** نخ‌ها فقط می‌توانند منابع را به **ترتیب صعودی شماره‌ها** درخواست کنند.
    - یعنی اگر منبع شماره ۵ را داری، دیگر حق نداری منبع شماره ۱ را بخواهی. (اگر ۱ را می‌خواهی، باید اول ۵ را آزاد کنی).

- **چرا این کار جلوی بن‌بست را می‌گیرد؟ (اثبات منطقی)**
    - فرض کنیم دایره‌ای ایجاد شود: T_1 منتظر T_2، ... و T_n منتظر T_1.
    - چون T_1 منتظر T_2 است، پس منبع در دست T_2 باید شماره‌اش **بزرگتر** از منبع در دست T_1 باشد.
    - پس: $F(R_1) < F(R_2) < ... < F(R_n) < F(R_1)$
    - نتیجه: $F(R_1) < F(R_1)$ که غیرممکن است! پس دایره تشکیل نمی‌شود.

📊 جدول جمع‌بندی (مخصوص شب امتحان)

وضعیت عملی بودن	روش پیشگیری (حمله)	شرط لازم
❌ **غیرممکن** (برای قفل‌ها و پرینتر).	اشتراکی کردن همه منابع.	**انحصار متقابل**
⚠️ **ممکن اما بد** (باعث قحطی و کاهش کارایی).	۱. گرفتن همه منابع اول کار. ۲. درخواست فقط با دستان خالی.	**نگهداری و انتظار**
⚠️ **محدود** (فقط برای CPU و دیتابیس خوبه).	اگر منتظر شدی، دارایی‌هات رو پس بده.	**عدم پس‌گیری**
✅ **بهترین و عملی‌ترین روش.**	**ترتیب‌دهی خطی به منابع (Ordering).**	**انتظار چرخشی**

🌟 نکات تکمیلی برای نمره ۲۰

- **جاوا (Java):** در جاوا برای جلوگیری از بن‌بست در synchronized، برنامه‌نویسان اغلب از System.identityHashCode استفاده می‌کنند تا یک ترتیب عددی برای قفل‌ها ایجاد کنند و همیشه به ترتیب قفل بگیرند.
- **تفاوت پیشگیری و اجتناب:** در "پیشگیری" ما قوانین محدودکننده می‌گذاریم (مثل همین ترتیب‌دهی) تا شرط‌ها نقض شوند. اما در "اجتناب" (بخش بعدی درس)، ما محدودیت نمی‌گذاریم، بلکه هر درخواست را بررسی می‌کنیم که امن است یا نه (مثل بانکدار).